Patient engagement through AI-driven chatbots

CS39620 Minor Project Report

Author: Artur Andrijauskas (ara38@aber.ac.uk)

Supervisor:
Praboda Rajapaksha (prr16@aber.ac.uk)

12th March 2024

This report is submitted as partial fulfilment of a BSc degree in
Computer Science and Mathematics (GG14)

Department of Computer Science
Aberystwyth University
Aberystwyth
Ceredigion
SY23 3DB
Wales, UK

Declaration of originality

I confirm that:

- This submission is my own work, except where clearly indicated.
- I understand that there are severe penalties for Unacceptable Academic Practice, which can lead to loss of marks or even the withholding of a degree.
- I have read the regulations on Unacceptable Academic Practice from the University's Academic Registry (AR) and the relevant sections of the current Student Handbook of the Department of Computer Science.
- In submitting this work, I understand and agree to abide by the University's regulations governing these issues.

Name Artur Andrijauskas

Date 06/04/25

Consent to share this work

By including my name below, I hereby agree to this project's report and technical work being made available to other students and academic staff of the Aberystwyth Computer Science Department.

Name Artur Andrijauskas

Date 06/04/25

Abstract

This project developed an AI-driven chatbot to enhance patient education and adherence to prostate cancer surveillance protocols. The chatbot was designed to address patient queries, deliver accurate medical information, and promote compliance with screening guidelines through natural, evidence-based interactions. A structured knowledge base was created to consolidate information on prostate cancer risk factors, symptoms, and treatment options, enabling the system to retrieve and present relevant data efficiently.

The work utilized Python alongside frameworks such as Flask for backend integration and ngrok for secure local-to-web deployment. Advanced conversational AI techniques, including prompt engineering and Chain of Thought (CoT) reasoning, were applied to improve the chatbot's ability to interpret complex patient inquiries and generate coherent, guideline-aligned responses. The development process emphasized iterative testing and refinement, with conversational flows dynamically adjusted based on feedback to ensure clarity and user engagement.

Key outputs include a functional chatbot capable of addressing common patient concerns and a modular knowledge repository adaptable for future expansions. The agile methodology as well as monitoring of progress through Notion guided the project, prioritizing incremental improvements and usability testing. Preliminary evaluations demonstrated the chatbot's effectiveness in reducing information gaps and fostering patient confidence in surveillance practices. By integrating user-centered design with scalable AI tools, this work highlights the potential of conversational agents to bridge communication barriers in healthcare, offering a foundation for deploying similar solutions in other medical domains.

Contents

1. BACKGROUND, ANALYSIS & PROCESS 6

1.1. Background 6

1.2. Analysis 6

1.3. Process 7

2. DESIGN 10

3. IMPLEMENTATION 18

4. TESTING 29

5. CRITICAL EVALUATION 31

6. REFERENCES 33

7. APPENDICES 34

A. Use of Third-Party Code, Libraries and Generative AI 35

1 Background, Analysis & Process

1.1 Background

This projects primary aim was to provide and improve patient engagement and education relating to prostate cancer through AI driven chatbots providing medically accurate and conversational style responses to medically based queries. This was a project that utilized and was split between both software engineering and research, thorough research needed to be completed to compare different models and frameworks for suitability for the project's objectives however technical work and programming was also necessary to ensure a seamlessly working API and product.
In preparation for the project as well as during the early stages of development numerous articles were covered to gain further insight and better understand the current space of AI chatbots in prostate cancer [1], [2], [3], [4], [5]. Diving in depth on the limitations and progressions when using AI chatbots aided future decisions of frameworks and models to better help resolve and address the key limitations and drawbacks.
Moreover, the excess free time and stress-free period at the start of the semester was utilized to gain a head start in the progress of the project. Thorough model and framework research was conducted in the initial weeks of the project commencing. This aided in the understanding and consolidation of a variety of possible ways to conduct the project which can be thought through and analysed over a longer period.
This project struck out and was of particular interest to work on as I have great passion for the current progress and applications of AI in today's world and want to pursue this innovative industry for both my future career and business.

1.2 Analysis

1.2.1 Impact on patients

Prostate cancer education and knowledge is a sensitive topic which is rarely shared nor gains much coverage amongst individuals whether it's in person or online. This lack of knowledge and education thus leads to patients with concerns or queries to be either misinformed or misinterpret their current situation leading to redundant and unnecessary screenings and appointments, further backlogging and increasing the demand and wait time for healthcare services. Vice versa, those with genuine demands for professional healthcare services are unaware of their situation's severity. In reference [5] it states that reasons for patients not getting screened for PC include obstacles to access, lack of knowledge, fear of cancer, embarrassment, and low perceived risk. These are all outcomes of poor discussion and acknowledgment of prostate cancer in general throughout individuals and online. Information about PC and PC screening is often received through lay media, friends, or family members which is often low-to-moderate quality and poorly actionable for health consumers [5]. Increasing numbers of searches relating to cancer treatment and screening is promising as more individuals are gaining confidence to address their health concerns however despite this patient trust in online resources is low, with 80% of participants reporting a trust level of some, a little, or not at all [5]. Patients are also finding it increasingly frustrating to get into contact with real healthcare professionals due to the scarcity of current healthcare services. A growing need for medically accurate and reliable online information is required and becoming a large demand for individuals looking for answers. Therefore, the project needed to address these main concerns by providing patients with

- Accurate, up to date and medically approved information about PC

- References to established and highly professional medical institutions and sources
- Fast and reliable responses to urgent medical queries and concerns
- Professional yet empathetic and conversational style responses
- Reminders to refer to real medical professionals

Addressing these key areas ensures that major aspects and limitations of current chatbot use are addressed to provide an improved patient experience regarding information and education to PC. This would certainly be possible with the completion time provided. Focusing, on key aspects such as a medically accurate database, a suitable models for our chosen conversational approach and appropriate prompt engineering techniques to provide well spoken and relevant responses.

1.2.2 Impact on doctors

Despite the growing need for online information about prostate cancer for patients, PC AI chatbots have shown to have a significantly positive impact when paired with real healthcare professionals to make high impact decisions. In reference [1] it is stated that chatbots on their own are outperforming doctors who could access only an internet search and medical references, but armed with their own LLM, the doctors, from multiple regions and institutions across the United States, kept up with the chatbots. This further reinforces the benefits and capabilities that chatbots have in the healthcare industry as their accuracy and precision to diagnose and determine treatments and cases exponentially improves without the possibilities of human error. In a medical context, decisions can get tricky. Say a doctor incidentally discovers a hospitalized patient has a sizeable mass in the upper part of the lung. What would the next steps be? The doctor (or chatbot) should recognize that a large nodule in the upper lobe of the lung statistically has a high chance of spreading throughout the body [1]. The addition of a secondary voice of the chatbot can help reinforce or challenge doctors decisions for their patients which ultimately Is a huge benefit as skilled doctors potentially may misdiagnose patients which can then be easily flagged up when paired with a chatbot. Likewise, a decision that a doctor is skeptical about can be reinforced and affirmed by a chatbot.

Implementing the chatbot as second voice rather than a co-worker not only increases output as more work is being addressed overall however may be more suitable in general. In an article, researchers tapped a group of board-certified doctors to create a rubric that would qualify a medical judgment or decision as appropriately assessed. The decisions were then scored against the rubric. To the team's surprise, the chatbot outperformed the doctors who had access only to the internet and medical references, ticking more items on the rubric than the doctors did [1]. Proving that the potential for chatbots to independently make smarter and more high impact decisions is growing at an increasing speed.

The question remains, when will this happen? For now, researchers and doctors believe that chatbots will continue to be a valuable asset when paired with medical professionals, developing improvements and iterations however look promising for AI in healthcare in the near future.

1.3 Process

1.3.1 Methodology

To effectively approach the project and have an organized and structured way of dealing with workload, new tasks, deadlines and feedback, an agile/scrum methodology was implemented to ensure the above. This methodology relied on sprints lasting 1-2 weeks, aligning well with supervisor meetings to create tasks and objectives which needed to be

tackled within a certain period. Moreover, implementing the methodology meant that iterative feedback and modifications could be made throughout the project to satisfy and meet new demands, priorities and concerns that arose. This methodology was crucial to the project as improvements of the chatbot were made continuously through experimentation of models, prompts and further techniques. RLHF is a learning technique that thrives well with this methodology as constant feedback for users lead to space for improvements and adjustments to further increase the chatbots capabilities. Therefore, the need to create sprints and stories based on feedback was crucial and a priority throughout the project and key decision factor for the methodology.

1.3.2 Other Software

The use of a software called Notion was used to better structure this project's overall workload and organization, not just the methodology. Figure 1 shows a dedicated page for the project that was made which held majority of the progress of the project.

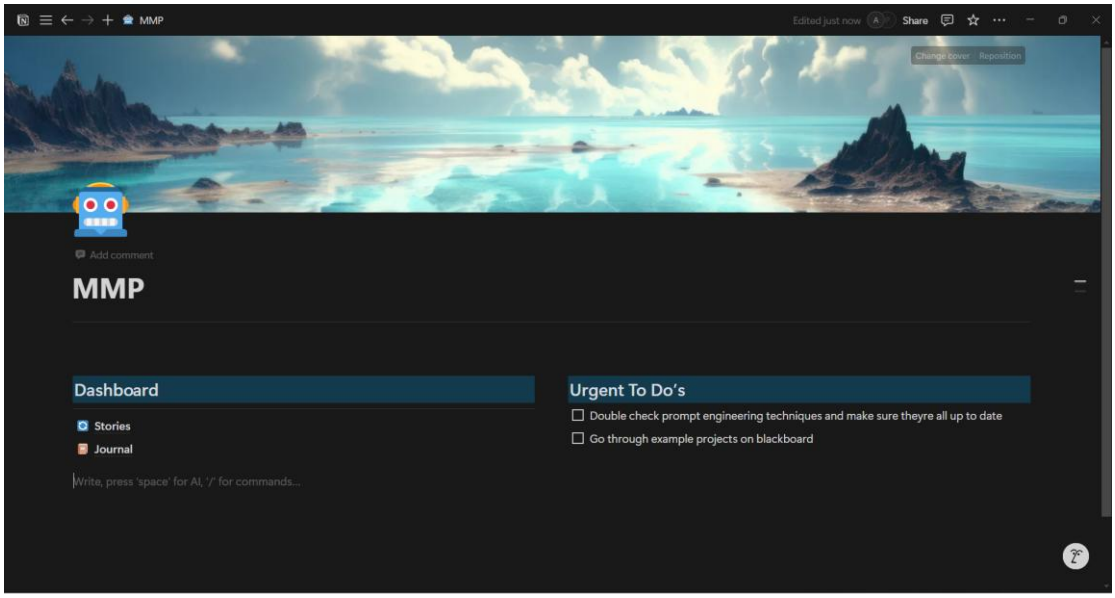


Figure 1

Figure 2 shows the journal page that was created to accurately track tasks and workload completed for each day in detail for a clear progression and workload completed indicator.

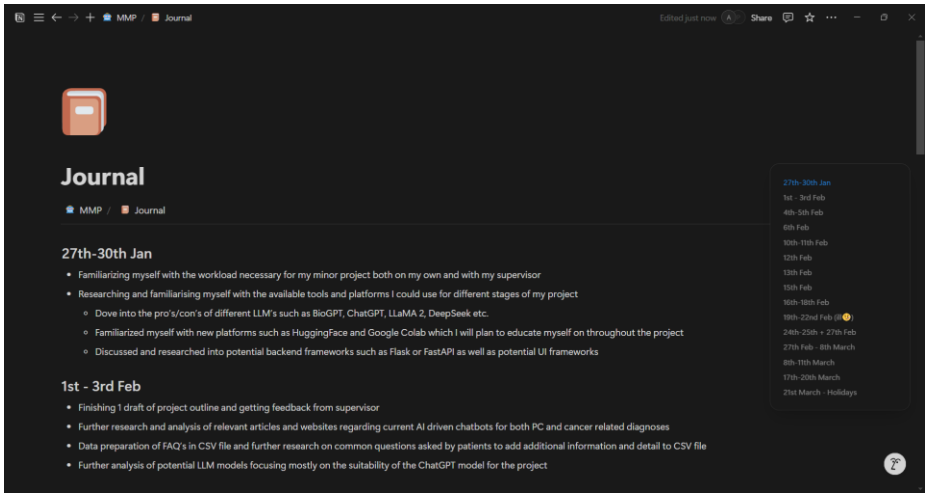


Figure 2

Lastly, Figure 3 shows stories and sprints that were stored on their own individual page on a table organizing them into their suitable category of progression. Upon selection of one of the pages, detail and analysis of each task and decision was noted and brainstormed on each stories individual page as well as in a physical notebook dedicated to the project

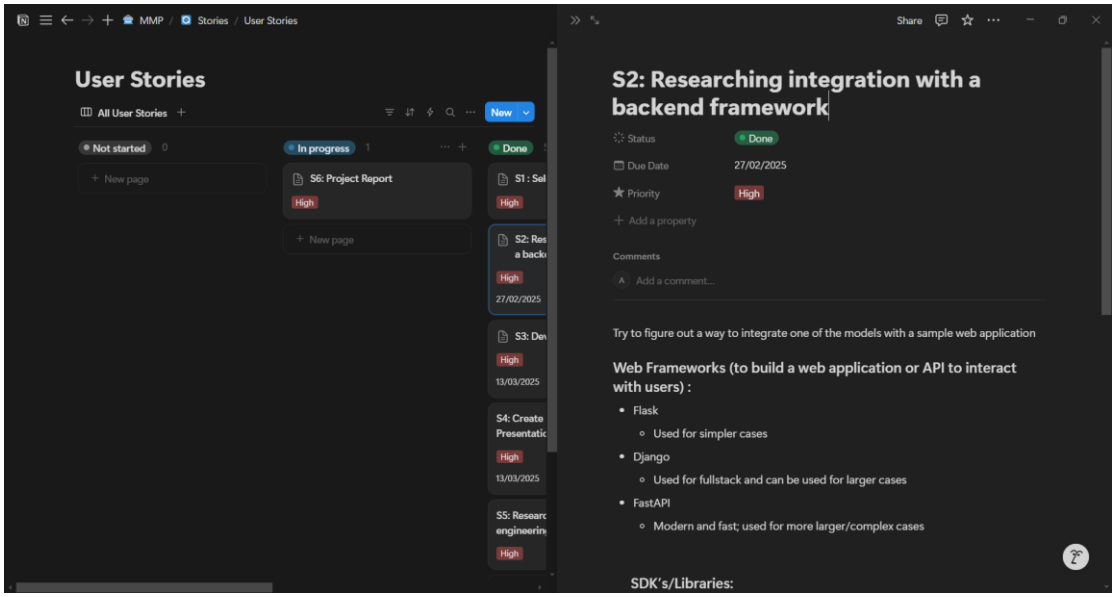


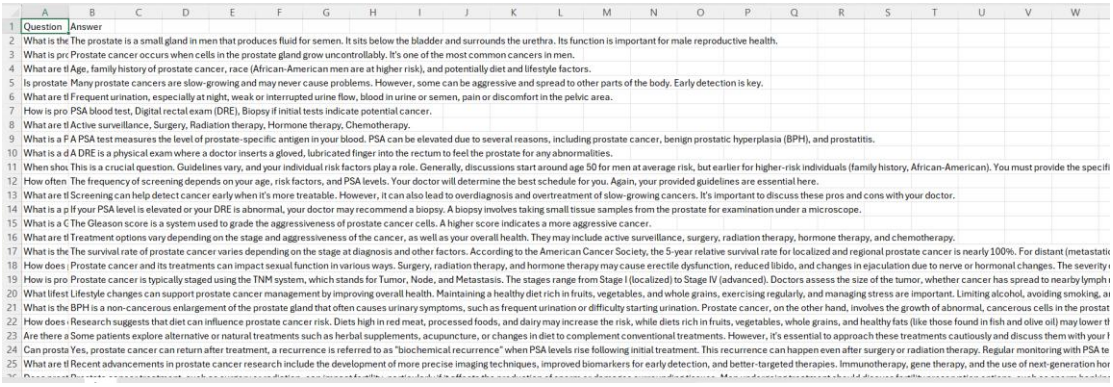
Figure 3

2 Design

2.1 Database Design

The initial step towards designing and creating this project was to set up an appropriate and suitable database for the model to be able to seamlessly retrieve medically accurate information seamlessly. As the scope of a Minor project is reduced compared to a Major project, a simpler approach was taken when constructing the database.

Figure 4 shows the structured CSV that was created to initially store medically accurate FAQ's which were provided in a format of Question and Answer. The simple premise of FAQ's and their structure would allow the model to more easily retrieve as well as differentiate patient queries from the database due to its simplistic design.



Question	Answer
What is the	The prostate is a small gland in men that produces fluid for semen. It sits below the bladder and surrounds the urethra. Its function is important for male reproductive health.
What is pro	Prostate cancer occurs when cells in the prostate gland grow uncontrollably. It's one of the most common cancers in men.
What are th	Age, family history of prostate cancer, race (African-American men are at higher risk), and potentially diet and lifestyle factors.
Is prostate	Many prostate cancers are slow-growing and may never cause problems. However, some can be aggressive and spread to other parts of the body. Early detection is key.
What are th	Frequent urination, especially at night, weak or interrupted urine flow, blood in urine or semen, pain or discomfort in the pelvic area.
How is pro	PSA blood test, Digital rectal exam (DRE), Biopsy if initial tests indicate potential cancer.
What are th	Active surveillance, Surgery, Radiation therapy, Hormone therapy, Chemotherapy.
What is a F	A PSA test measures the level of prostate-specific antigen in your blood. PSA can be elevated due to several reasons, including prostate cancer, benign prostatic hyperplasia (BPH), and prostatitis.
What is a d	A DRE is a physical exam where a doctor inserts a gloved, lubricated finger into the rectum to feel the prostate for any abnormalities.
When shou	This is a crucial question. Guidelines vary, and your individual risk factors play a role. Generally, discussions start around age 50 for men at average risk, but earlier for higher-risk individuals (family history, African-American). You must provide the specifi
How often	The frequency of screening depends on your age, risk factors, and PSA levels. Your doctor will determine the best schedule for you. Again, your provided guidelines are essential here.
What are th	Screening can help detect cancer early when it's more treatable. However, it can also lead to overdiagnosis and overtreatment of slow-growing cancers. It's important to discuss these pros and cons with your doctor.
What is a p	If your PSA level is elevated or your DRE is abnormal, your doctor may recommend a biopsy. A biopsy involves taking small tissue samples from the prostate for examination under a microscope.
What is a G	The Gleason score is a system used to grade the aggressiveness of prostate cancer cells. A higher score indicates a more aggressive cancer.
What are th	Treatment options vary depending on the stage and aggressiveness of the cancer, as well as your overall health. They may include active surveillance, surgery, radiation therapy, hormone therapy, and chemotherapy.
What is the	The survival rate of prostate cancer varies depending on the stage at diagnosis and other factors. According to the American Cancer Society, the 5-year relative survival rate for localized and regional prostate cancer is nearly 100%. For distant (metastatic
How does	Prostate cancer and its treatments can impact sexual function in various ways. Surgery, radiation therapy, and hormone therapy may cause erectile dysfunction, reduced libido, and changes in ejaculation due to nerve or hormonal changes. The severity
How is pro	Prostate cancer is typically staged using the TNM system, which stands for Tumor, Node, and Metastasis. The stages range from Stage I (localized) to Stage IV (advanced). Doctors assess the size of the tumor, whether cancer has spread to nearby lymph
What lifest	Lifestyle changes can support prostate cancer management by improving overall health. Maintaining a healthy diet rich in fruits, vegetables, and whole grains, exercising regularly, and managing stress are important. Limiting alcohol, avoiding smoking, a
What is the	BPH is a non-cancerous enlargement of the prostate gland that often causes urinary symptoms, such as frequent urination or difficulty starting urination. Prostate cancer, on the other hand, involves the growth of abnormal, cancerous cells in the prostat
How does	Research suggests that diet can influence prostate cancer risk. Diets high in red meat, processed foods, and dairy may increase the risk, while diets rich in fruits, vegetables, whole grains, and healthy fats (like those found in fish and olive oil) may lower th
Are there a	Some patients explore alternative or natural treatments such as herbal supplements, acupuncture, or changes in diet to complement conventional treatments. However, it's essential to approach these treatments cautiously and discuss them with your h
Can prosta	Yes, prostate cancer can return after treatment, a recurrence is referred to as "biochemical recurrence" when PSA levels rise following initial treatment. This recurrence can happen even after surgery or radiation therapy. Regular monitoring with PSA te
What are th	Recent advancements in prostate cancer research include the development of more precise imaging techniques, improved biomarkers for early detection, and better-targeted therapies. Immunotherapy, gene therapy, and the use of next-generation hor

Figure 4

To further enhance the quality and variety of FAQ's provided, further research was conducted to explore other frequently asked patient queries. The addition of more researched queries would allow a broader range of discussion and engagement from the chatbot when working with patients. This would further encompass a larger range and variety of concerns which ultimately would lead to increased satisfaction and information provided to patients.

2.2 Work Environment

2.2.1 Selection

Before tackling major project objectives, an appropriate work environment and IDE was needed for development to take place in. Initial consideration was taken towards using PyCharm or VS Code as they're both extremely well-functioning IDE's and environments which I've had plenty of experience using in the past. Despite this unfortunately, the desktops power was needed to be taken into great consideration due to the nature of the project. With loading, fine tuning and generating accurate and precise responses from various models, a stronger laptop/desktop would've been needed to accommodate the high performance and power demands from the models.
To resolve this problem, development of the project was decided to be completed on Google Colab. Google Colab is a free cloud service that allows individuals to create and run Jupyter notebooks with access to GPUs and TPUs, ideal for projects where high computing

power is necessary but not available locally. Google colab is beneficial for training or running complex AI models and eliminates the need for environment setup since many libraries are pre-installed. Colab also allows for easy collaboration and sharing, with notebooks stored in the cloud.
However, it comes with some drawbacks. An internet connection is required, and there are limitations on resources such as RAM and session durations. Additionally, individuals have less control over the environment compared to a local IDE as quick edits or debugging can feel slower than working directly in a local setup. Moreover, if the project was of larger scope and incorporated real patient data and information, storing data or code in the cloud may raise security concerns.

2.2.2 Impact on Model selection

The use of Google Colab meant that too large of a model size ran the risk of not being able to run on the software without purchasing better facilities, meaning that a smaller model would be more appropriate to use with the software. This meant that our model was more likely to be specialized and fine-tuned to data or information, generating more precise responses within a smaller range of topic, rather than a more generalist response being able to explore a variety of discussions. This would serve well for the objectives of the project as accurate and precise medical information and responses are necessary for patients however if the project was of larger scope, consideration of a larger model would've been considered due to my personal interests of more generalist and conversational style of approaches in AI chatbots.

2.3 Model Analysis

One of the most important design and analysis elements of the project was the selection of the chatbot model. The objectives of the project ensured that the priorities when selecting our model lay in the model's reliability to provide medically accurate information and advice for patients whilst always ensuring references to real world professionals when such queries attained a more personal approach with more specific details and expectations of possible diagnoses.
To achieve this criterion, research was conducted from a variety of sources which covered popular models being used and leveraged to achieve and train high quality models in the healthcare industry. These models were taken from Hugging Face, a reliable and well-known software containing numerous popular open-source models.

2.3.1 General Purpose Models

General purpose models were first addressed in selection; models of interest were ChatGPT, Deepseek and LLAMA. These models overall excel in their ability to be conversational whilst having the ability to formulate responses to a variety of topics and conversations. This is a factor that was highly considered as specificity and precise accuracy is a primary objective during selection of the chatbot model. Therefore, there's a potential risk of more generalisation at the cost the accurate information when using these models for healthcare and medically based applications, especially at smaller parameters.

2.3.1.1 ChatGPT

In article [4] a study was conducted to interpret the quality of responses from ChatGPT when paired with data snippets compared to not. The model demonstrated an ability to formulate interpretable responses, which were similar in quality and content to Google's feature snippet. Arguably, the additional context often provided by ChatGPT appeared to minimize the likelihood of alarm while providing action items.
Nonetheless, for the question that Google did not produce a snippet, ChatGPT performed poorly, which may in part be resultant of the question wording. This therefore signals that poor quality of responses may occur when not provided with relevant context and information. This is highly impactful as the structured database of FAQ's may occasionally not address certain patient queries and thus leaves the model at risk of providing false or misleading information.
It's safe to assume this research was also conducted on a larger parameter model (3B+), further reinforcing that due to our small project scope, if an even smaller GPT model was to be ran, an even larger decrease in response quality would be present due to its limited knowledge base.

However to still consider a variety of models, openai-community/gpt2, a model with parameter size of 137M, was tested on Colab due to its suitable size. Figure 5 shows Colab's performance demands when running this model through a simple block of code, clarifying the model's suitability for Colab.

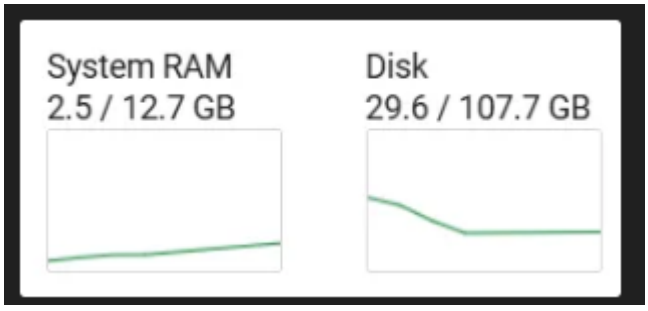


Figure 5

2.3.1.2 DeepSeek

Similarly, consideration and analysis of the general-purpose model deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B was conducted to see how it could possibly have a positive impact on current healthcare industries. This model was much larger in parameter size than the GPT model however testing on Colab was necessary to test the boundaries of the software and experiment with different sized models in general. Testing showed an increase in response time as well as the need for an additional GPU. The models need for more time and computing power to generate responses can potentially lead to a decrease in patient satisfaction due to longer response times.
Furthermore, the lack of research of current implementation of Deepseek in the healthcare industry was a factor to consider with this model as its current known strengths lay in its latest models where they're known for their ability to perform better at logical arithmetic such as coding and math problems.

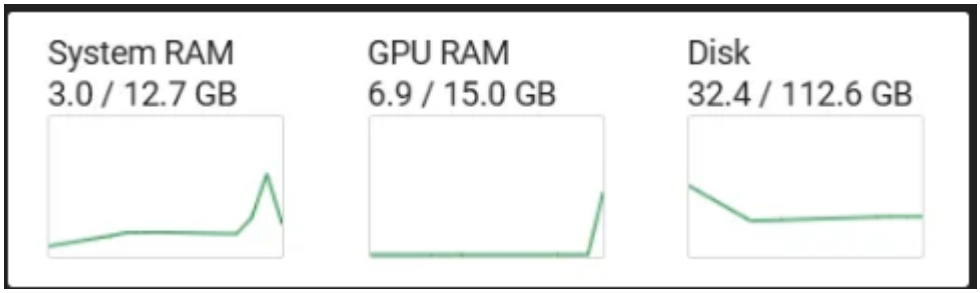


Figure 6

2.3.1.3 LLAMA Models

The final general-purpose model assessed was meta-llama/Llama-3.2-1B. A key advantage of LLAMA 3.2 is its lightweight architecture allows for fast inference speeds and low computational requirements, making it ideal for running on Google Colab or in similar offline/local environments where access to powerful hardware or the internet might be limited. This is especially useful in healthcare settings where reliability, responsiveness, and patient data privacy are critical.

A reading level test was conducted of 8th grade or less in 0 (0 %) of the responses from ChatGPT, 1 (17 %) from Co-Pilot, 2 (33 %) from Gemini, 3 (50 %) from Meta AI, 2 (33 %) from Claude, 2 (33 %) from Perplexity, and 0/6 (0 %) from Grok [2]. Potentially showing LLAMA's capabilities to best formulate responses amongst popular models.

However, a major downside to these general-purpose models is that they don't come pre-trained on medical knowledge, so training and finetuning on specifical medical data is needed to ensure accurate and relevant medical responses. LLAMA also lacks built-in safety features, meaning development of safeguards may be necessary to prevent generation of harmful or misleading medical responses.

Despite its slightly larger size compared to previous models, Figure 7 shows that LLAMA still managed to successfully run on Google Colab without necessary external computing power.

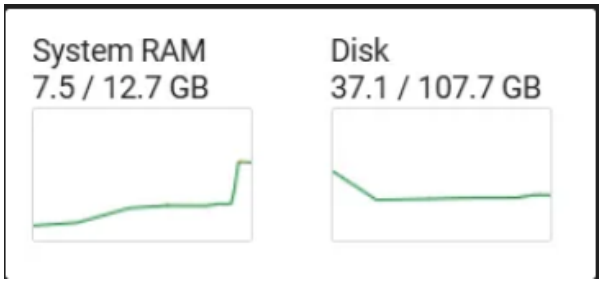


Figure 7

Concluding analysis of these popular general-purpose models showed that these models have the potential to positively influence healthcare through chatbot applications if implemented at a larger scale due to their suitability and lenience towards larger parameter models for more general and broader discussions regarding healthcare. Despite this, they were not appropriate for the objectives of the project. Precise, specialized and accurate medical information was a priority thus models specifically tuned or developed for the purpose of healthcare information or relevancy to the healthcare industry were models that would be more suitable for our chatbot.

However, consideration of implementation of one of these models was still possible if the project came to meet certain criteria such as over specialization and inability to comprehend basic medical related queries not requiring heavy medical information.

2.3.2 BioGPT

Promising research was conducted on the 347M parameter model, BioGPT. BioGPT is a specialized generative language model that was trained on scientific papers and medical data, making it well-suited for generating accurate and relevant responses to health-related queries. This also enables it to grasp complex medical terminology and concepts better than general-purpose models. Moreover, BioGPT was evaluated on six biomedical natural language processing tasks and outperformed previously tested models on most tasks. The model achieved a rate of 44.98%, 38.42% and 40.76% F1 score on BC5CDR, KD-DTI and DDI end-to-end relation extraction tasks, respectively, and 78.2% accuracy on PubMedQA, setting a record amongst model performance [9].

However, its disadvantages lie in its limited scope of general conversational ability, as it may struggle with everyday language, empathetic responses, or broader conversational context outside of the biomedical domain. Moreover, smaller models such as BioGPT have fewer parameters, meaning they can't store or represent as much information about the world or a specific domain. This therefore means if there are any gaps in knowledge or topics which aren’t covered by the model, hallucinations may occur. This is less common in larger models as more broad information is available to them therefore, for BioGPT, its potential for hallucinations due to lack of information was highly considered when selecting our model.

2.3.3 Embedding/Retrieval Model

Appropriate selection of an embedding model was necessary to ensure reliable and trusted retrieval of relevant information from the data.csv. The model’s ability to correctly classify the users medically related query to the appropriate FAQ + answer was a necessary implementation to be able to formulate a relevant and informative response for the user. Numerous potential models from Hugging Face, primarily BERTs, were researched on their suitability within the project. Models such as Bio_ClinicalBERT, SapBERT-from-PubMedBERT-fulltext, distilbert-base-uncased and more were researched and tested within our future chatbot.

2.3.4 Classification Model

A simple classifier was needed to determine whether the users query was medically related. The zero-shot classifier, facebook/bart-large-mnli, was tested in its ability to determine whether a query was medically related. It eliminated the need for labeled training data on queries by allowing classification of text based on natural language descriptions like "medical" and "non-medical." This flexibility makes it easy to adapt or expand categories without retraining if further expansion was considered. Because the model is trained on natural language inference, it understands context and semantics rather than relying on simple keyword matching, enabling it to identify medical relevance even in less explicit queries. It also required minimal setup and simple integration with the transformers library, making it a powerful, generalizable, and easy-to-implement solution for early-stage applications or prototyping where speed and adaptability are key.

2.4 Web Framework

Detailed analysis and research was done to identify suitable web frameworks which would comply and appropriately suit the project at hand.

2.4.1 Flask

Initial consideration was for Flask, its simplicity and lightweight design, creates opportunity to build a minimalistic healthcare chatbot without unnecessary bloat. Although out of scope, modularity allows components to be handpicked such as authentication or database integration. This flexibility is ideal if the chatbot requires custom security protocols upon further expansion, such as end-to-end encryption for patient interactions. However, Flask's lack of built-in features meant if expansion of the chatbot was decided upon, manual implementation of critical components like rate limiting, user authentication, or API security headers would need to be implemented, which could introduce risks if not rigorously tested. For example, relying on third-party plugins for encryption or audit logging might expose vulnerabilities if those libraries are outdated or poorly maintained. Flask's model was an ideal candidate due to its small and simple design which could be easily integrated within our chatbot project scaling the chatbot to handle high concurrency, such as during peak healthcare inquiry periods, could become challenging without additional tooling.

2.4.2 FastAPI

FastAPI was the secondary consideration, a web framework designed for high performance, leveraging Python 3.7+ async features and Pydantic for data validation. FastAPI allows the ability to automatically generate interactive API documentation from your type-annotated code, speeding up both development and possible client integration in established healthcare systems/software. Additionally, its async support allows handling large numbers of concurrent requests with minimal overhead. Despite this, FastAPI is a relatively new framework with a smaller community and fewer mature third-party solutions; its dependency on type hints and specific syntax can present a steeper learning curve, and projects may suffer from organization bloat in a single main file if not carefully structured. This ensures that the web framework holds its strength within its modern and rapid ability however for the projects use case, may be slightly inappropriate due to its preferred nature of being used to address larger and more complex tasks as well as steeper learning curve which may be a hindrance within the time span given for this project.

2.4.3 Django

Final consideration was done on the Django full-stack framework, a framework offering an ORM, authentication, admin interface and templating. Django's "batteries-included" approach is appealing for its robust security features and rapid development potential. Built-in protections against threats like SQL injection, XSS, and CSRF would reduce the risk of data breaches which are non-negotiable for a healthcare chatbot handling sensitive patient information and data. This philosophy also allows the acceleration of development of CRUD-heavy applications and ensures consistent conventions across projects, backed by a large, active community and comprehensive documentation. This reinforces the fact that Django may be a framework more suitable for larger and more complex projects, like FastAPI which therefore is slightly unsuitable for the smaller project size. It's monolithic architecture and many built-in abstractions can be overkill for simple chatbots, leading to slower initial setup,

a steeper learning curve, and potential performance overhead for smaller workloads such as this.

2.5 SDKs/Libraries

2.5.1 Hugging Face Transformers

Hugging faces well-known transformers library was easily considered to be a very useful tool to utilize for the project. Leveraging Hugging Face Transformers for its pre-trained models would enable accurate symptom parsing and response generation. The library's pipelines accelerate prototyping due to its simplistic and reliable nature as well as its open-source community aids troubleshooting. However, deploying larger models would require costly computational resources and using cloud-based APIs risks violating HIPAA/GDPR compliance unless self-hosted. Additionally, safeguards against model hallucinations, such as cross-referencing outputs with verified medical databases or referring to healthcare professionals, are essential factors which need addressing within the project to prevent misleading advice.

2.5.2 PyTorch

PyTorch was an ideal deep learning framework for this healthcare chatbot project due to its simplicity and seamless compatibility with Hugging Face Transformers. It allows for easy integration with powerful pre-trained models such as BioGPT and LLAMA, enabling efficient development of natural language processing tasks like medical query understanding and response generation. PyTorch also offers reliable GPU acceleration, which improves training and inference speed, allowing the chatbot to respond quickly and handle real-time interactions smoothly. Its straightforward development process makes it easier to test and fine-tune models, ultimately supporting the creation of a responsive, scalable, and medically specialized chatbot that can assist users with health-related concerns accurately and efficiently.

2.5.3 Other tools

SQLite3 was a necessary implementation to create a reliable and stable connection between our API and the database's. The demand for a coherent tool that enables seamless addition and adjustments of database content meant that SQLite3 was a tool that was most suitable and appropriate for future human feedback to be easily captured and permanently stored within a database.

The pandas library was also used due to its powerful and efficient data manipulation capabilities. Pandas allowed simple loading and processing of structured data from CSV files and SQLite databases, such as frequently asked questions and user feedback, which are critical for training the chatbot and evaluating its performance. Figure 8 shows that pd.read_csv() was used to extract questions and answers from a dataset, enabling seamless integration with the embedding model for response generation. Additionally, Figure 8 shows how pd.read_sql() was utilized to retrieve feedback records, helping the monitoring and refinement of the chatbot based on user interactions. Pandas' intuitive syntax and versatile functionality made it an ideal choice for managing and analyzing the tabular data essential to the healthcare chatbot.

```
faq_path = f'{drive_path}/data.csv'
faq_df = pd.read_csv(faq_path)
questions = faq_df['Question'].tolist()
answers = faq_df['Answer'].tolist()

question_embeddings = embedding_model.encode(questions, convert_to_tensor=True)
```

(parameter) convert_to_

```
# Be able to view last 10 queries and their feedback
def view_feedback():
    db_path = f'/content/drive/My Drive/MedBot_Data/feedback.db'
    conn = sqlite3.connect(db_path)
    df = pd.read_sql("SELECT * FROM feedback", conn)
    conn.close()
    return df

view_feedback().tail(10)
```

Figure 8

2.6 Overview

Figure 9 below displays a simplified depiction and overview of the flow of information and the relationships within the system. This was created briefly before the end of the project, hence why some parts and sections and their functionalities aren't shown. Despite this, they're addressed in detail within the rest of the report.

System Architecture of the Medical Chatbot Integrating Embedding, Classification, and Generative Models with User Feedback Loop

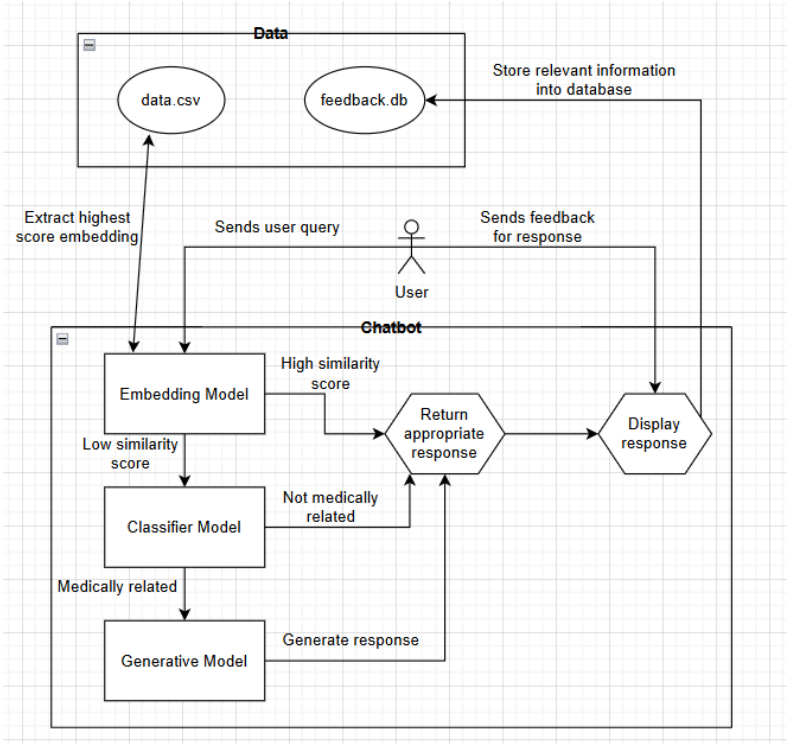


Figure 9

3 Implementation

3.1 Data

Initial implementation was focused on structuring and organizing colab to work seamlessly with external files from Drive such as data.csv, including all relevant FAQ-answers, as well as feedback.db, a database was implemented at a later stage, storing human feedback for every query-response pair for the model to be able to implement reinforcement learning. Figure 10 shows the initial integration of google drive and connection with our data.csv file.

```
import os
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
drive_path = '/content/drive/My Drive/MedBot_Data'
os.makedirs(drive_path, exist_ok=True)

if not os.path.exists(f'{drive_path}/data.csv'):
    raise FileNotFoundError(f"Please upload data.csv to {drive_path} via Google Drive website")
```

Figure 10

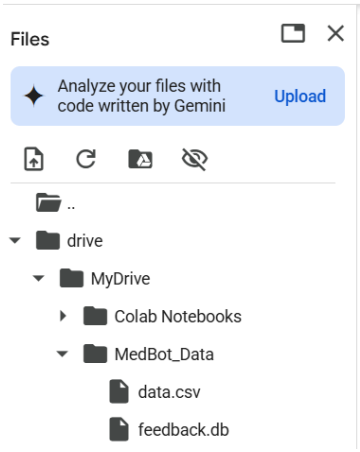


Figure 11

Connecting Colab to drive allowed access to files which would be permanently stored in drive compared to Colab which would be discarded every time the session was re-ran which is not suitable for our objective of potential reinforcement learning through human feedback, which requires large amounts of data collected over a long period of time. Figure 11 shows the side panel on Colab displaying current connected files and folders from Google Drive.

3.2 Generative Model Implementation

3.2.1 Early Stage

Figure 12 displays the first steps taken towards basic response generation which was implemented to test and analyse the initial prototype development models. Both BioGPT and LLAMA were primarily tested and configured during early development of the models with constant iterations and changes being made as more intricate techniques and processes were added such as prompt engineering and context handling.

```
import torch
from transformers import BioGptTokenizer, BioGptForCausalLM
from transformers import pipeline, set_seed, AutoTokenizer, AutoModelForCausalLM
```

Figure 12

Initially, testing proved that LLAMA had higher quality and better responses over BioGPT when faced with basic text generation with no prompt engineering. Both common and specialized queries were given to the models and LLAMA had consistently performed better, providing better formulated responses as well as less occurrence of hallucinations. This was not surprising as LLAMA was currently a general-purpose model with 1.5B parameters compared to a medically specialized model, BioGPT, of 347M parameters.
This however did not mean BioGPT was discarded as no longer a possible suitable model but instead stored as a potential model of use and comparison for later implementation. When further and more technical techniques are applied, they may prove to expose underlying weaknesses and strengths in both models that haven't yet been discovered and considered at these earlier stages of development.

```
tokenizer = BioGptTokenizer.from_pretrained("microsoft/biogpt")
model = BioGptForCausalLM.from_pretrained("microsoft/biogpt")

sentence = "What is Covid-19?"
inputs = tokenizer(sentence, return_tensors="pt")

set_seed(42)

with torch.no_grad():
    beam_output = model.generate(**inputs,
                                 min_length=100,
                                 max_length=1024,
                                 num_beams=5,
                                 early_stopping=True
                                 )
tokenizer.decode(beam_output[0], skip_special_tokens=True)
```

Figure 13

Figure 13 shows the structure the response generation took form in. This structure was also identically implemented for the LLAMA model to generate simple responses.

Despite the current preference for LLAMA over BioGPT at this early stage, a clear withdrawal of the LLAMA model lay in its response time. BioGPT averaged a response in roughly 5 seconds or less whilst LLAMA averaged a response time of roughly 20 seconds. This was a clear negative which needed to be addressed further along the project.

3.2.2 Late Stage

As development and iterations of prototypes progressed, the final most suitable generative model was decided to be BioGPT.
Note. Analysis of how this decision was made is discussed further in the following sections. This is section is strictly the discussion of the adjustments made directly towards the generative model.

The first section of Figure 14 below shows the tokenization applied to the prompt, Figure 24, using the BioGPT tokenizer, ensuring the input is truncated to a maximum length of 1024 tokens and returned as PyTorch tensors as well as a manually set seed for reproducibility.

The second section of Figure 14 displays the use of torch.no_grad() to disable gradient calculations for faster inference as well as choice of parameters which were informed by the model's domain-specific training and the need to balance coherence with variability. Max_new_tokens was set to 256 to keep responses informative yet concise, reflecting the typically focused nature of biomedical queries. The temperature=0.7 and top_p=0.9 were selected to introduce a moderate level of randomness, allowing the model to generate slightly varied but still medically sound answers without deviating into unrelated content. Since BioGPT was pre-trained on scientific data, these values help prevent overly rigid or overly

creative outputs. Do_sample was set to True to ensure the chatbot doesn't always generate identical responses to similar prompts, which helps improve the conversational feel. The repetition_penalty=1.2 discourages the model from repeating phrases, which can happen more often in domain-specific models like BioGPT that prioritize terminology precision. Lastly, decoding and splitting the output on “Response:” ensures a clean and direct answer for the user, enhancing clarity and usability. Each parameter was carefully chosen to optimize BioGPT’s strengths while minimizing potential issues like hallucinations.

```
inputs = tokenizer(
    prompt,
    return_tensors="pt",
    max_length=1024,
    truncation=True,
    add_special_tokens=True
)

set_seed(42)

with torch.no_grad():
    outputs = model.generate(
        **inputs,
        max_new_tokens=256,
        temperature=0.7,
        do_sample=True,
        top_p=0.9,
        repetition_penalty=1.2,
        num_return_sequences=1,
        pad_token_id=tokenizer.eos_token_id
    )

full_response = tokenizer.decode(outputs[0], skip_special_tokens=True)
return full_response.split("Response:")[-1].strip()
```

Figure 14

3.3 Web Integration

3.3.1 Web Framework Development

With working and functional responses, the next step was to construct and integrate the model into a suitable UI for the user to interact with the chatbot.
To achieve this, Figure 15 shows how flask was first imported to create and manage the backend server that handles chatbot requests and responses.

```
from flask import Flask, request, render_template_string, jsonify

app = Flask(__name__)
```

Figure 15

Once the flask application instance had been created, Figure 16 displays how a backend server endpoint (/) was created to receive user input via a POST request, process it with an AI chatbot function, and return a generated response in JSON format. When integrated with a frontend, such as an HTML page with a text input and a "Send" button, Java can capture the user’s query, send it to this route, and display the chatbot’s reply dynamically on the page. This connection is necessary to allow real-time communication between the user and the chatbot, where Flask handles the logic and response generation, while the HTML and JavaScript handle the user interface.

```python
@app.route("/", methods=["GET", "POST"])
def home():
    if request.method == "POST":
        user_input = request.form["user_input"]
        response = generate_response(user_input)
        return render_template_string(HTML_TEMPLATE, response=response, user_input=user_input)
    return render_template_string(HTML_TEMPLATE)
```

Figure 16

3.3.2 Initial HTML Template Creation

After initialization of our backend was completed, focus shifted on finding and adjusting a suitable HTML template for the chatbot. Due to the scope and objectives of the project, developing a HTML template from scratch was not a key focus or need therefore a pre-made online chatbot HTML template was found to facilitate this.
This was initially a very simplistic HTML template with quite a dull UI however it was sufficient for this stage of the project. A title, user query box, chatbot response and send button were all present and all that was required for the first prototype.
Figure 17 shows adjustments that were made to the HTML to properly facilitate and utilize flasks backend server by implementing appropriate POST methods and suitably be able to retrieve the chatbot response from the application.

```html
<form method="POST">
    <input type="text" name="user_input" placeholder="Ask me about prostate cancer..." required>
    <input type="submit" value="Get Answer">
</form>

{% if response %}
<div class="response">
    <h3>Here's what I found:</h3>
    <p>{{ response }}</p>
    <div class="disclaimer">
        ℹ️ Note: This AI assistant provides general information only. Always consult a healthcare professional for medical advice.
    </div>
</div>
```

Figure 17

A slight ahead of time modification was also made for the chatbot to always display a disclaimer for users to consult a healthcare professional every time a response was generated. This is to ensure understanding and compliance with healthcare guidelines and regulations and reinforcing that the chatbot is not responsible for personal and specialized advice for topics such as diagnoses and treatment recommendation.

3.3.3 Ngrok integration

Figure 18 shows how ngrok was implemented to create a secure tunnel from the internet to our local machine, allowing the locally running Flask server to be exposed to the web using a temporary public URL. The Flask server ran locally, which served both the backend logic and my premade HTML template that formed the chatbot's user interface and by running ngrok http 5000, a generated public URL was created to be able to forward web traffic to the Flask server, enabling users to interact with the chatbot from anywhere if shared. This meant potential for online testing would be possible assuming the server was continuously being ran on the local device.

```
ngrok_tunnel = ngrok.connect(5000)
print("Public URL:", ngrok_tunnel.public_url)
app.run(host='0.0.0.0', debug=False)
```

Figure 18

3.3.4 Future HTML template adjustment

Coming closer to the end of the project and further along the development process after numerous prototypes, focus was momentarily shifted on expansion of the UI to facilitate a better user experience when interacting with the chatbot. A more appealing HTML template, shown in Figure 19, was found and some minor preference adjustments were made. Below is a screenshot taken near the end of the project timeline showing the current UI.

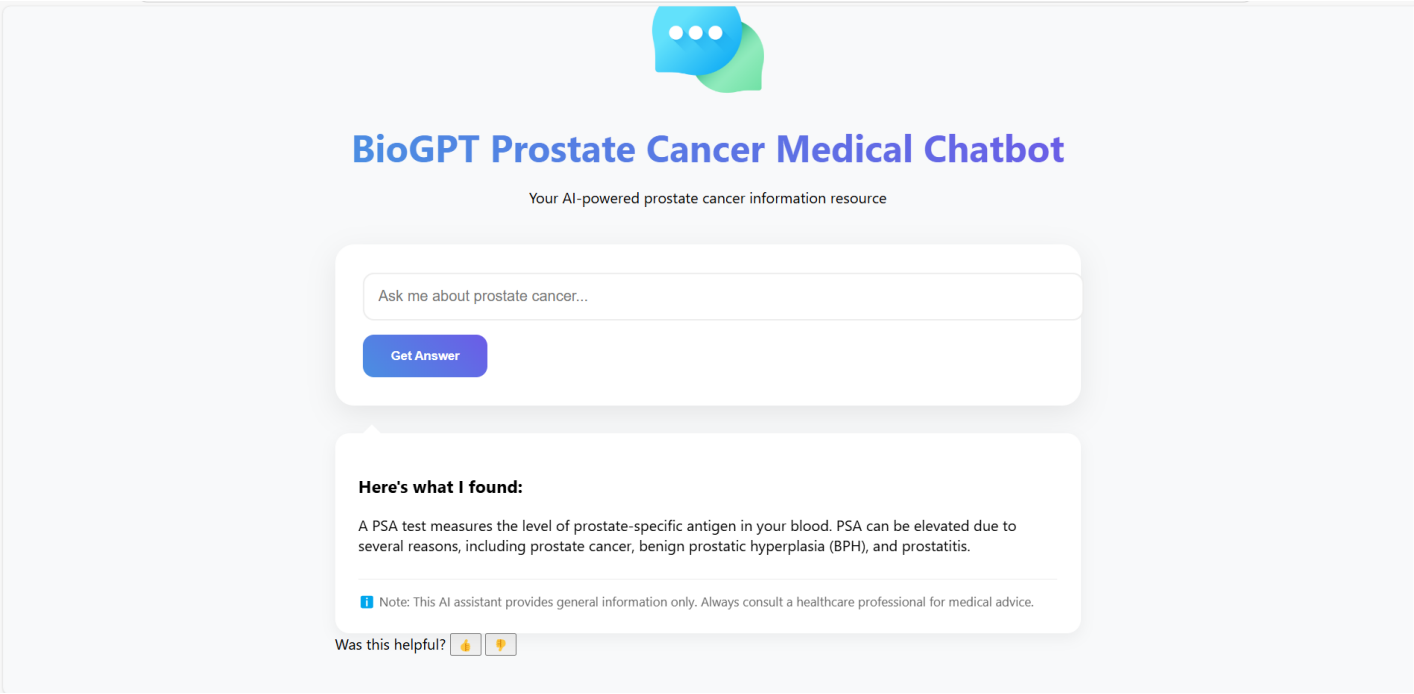


Figure 19

3.4 Retrieval Model Implementation

Next steps within the project included the implementation of the retrieval model to obtain and feed relevant and accurate data from data.csv based on the user query. Figure

Firstly, testing was done on a variety of BERT models for appropriate context retrieval. Figure 20 shows how importing sentenceTransformer from the transformers library allowed easy integration of models into the code.

```
embedding_model = SentenceTransformer('cambridgeltl/SapBERT-from-PubMedBERT-fulltext')
```

Figure 20

Secondly, embeddings had to be created for our data within data.csv for our retrieval models to be able to access, view and use the data to calculate future embedding scores. Figure 21 below displays how pandas was utilized to feed our database's path into our API and use in-built functions to store the relevant fields information.
The questions were then fed through our embedding models.encode() function which stored each individual question into their own appropriate embedding.

```
faq_path = f'{drive_path}/data.csv'
faq_df = pd.read_csv(faq_path)
questions = faq_df['Question'].tolist()
answers = faq_df['Answer'].tolist()

question_embeddings = embedding_model.encode(questions, convert_to_tensor=True)
```

Figure 21

Within the generate_response function, a need to calculate the similarity between user_query and question embedding was needed to be able to retrieve the most relevant piece of information for the database. Figure 22 shows how the user_input (user_query) is fed into the same function to facilitate an input_embedding. Both of which the user_input and all question_embeddings are fed into PyTorch's cosine similarity function which determines the similarity score of each question to the user_query.
The question which had the highest similarity score to the user_query had its index retrieved and the corresponding answer stored into a variable called context.

```
def generate_response(user_input, threshold=0.75):
    input_embedding = embedding_model.encode(user_input, convert_to_tensor=True)
    cos_scores = util.pytorch_cos_sim(input_embedding, question_embeddings)[0]

    top_score = torch.max(cos_scores).item()
    top_idx = torch.argmax(cos_scores)
    context = answers[top_idx]

    if top_score > threshold:
      return context
```

Figure 22

Figure 22 also displays that if the similarity score of the user_query was above a certain threshold the function would simply return the context, the relevant answer to the question retrieved. This meant that if the user query was very similar to a question already answered within the database, the answer to that question would simply be returned. This required iterative testing of threshold values to facilitate the balance of appropriate responses returns when similarity was very high. Too high of a score led to no return in context although the user query was essentially identical, thus need for generation and hence risk of inaccurate generation response. Too low of a score would risk the chatbot returning context to a unrelated query, also leading to misinformation or irrelevant responses.

After iterative adjustments and developments between a variety of BERT models and thresholds were conducted, the SapBERT model had shown the best results and accuracy with retrieving relevant data and a stronger suitability and accuracy when paired with the threshold value of 0.75 compared to other BERT models.

3.5 Prompt Engineering

A strong focus within the project was on the prompt engineering techniques used and fed through the model to modify, adjust and tune responses. At this stage in the project both BioGPT, a specialized and medically fine-tuned precise model trained on a small number of parameters, and LLAMA, a general-purpose model of 1.5B parameter size, were both being iteratively tested and applied and it is at this stage at which a final decision for a model was made.

Initial research conducted had shown that a variety of prompt engineering techniques could be applied to these models to facilitate and formulate more medically accurate and suitable responses. Role-playing was a primary technique considered and implemented by feeding the model a prompt to pretend or instruct it to act as a certain character.

```
prompt = f"""
You are Dr. Taylor, a virtual oncologist specializing in prostate cancer. Your goal is to provide clear, evidence-based, and compassionate answers to user questions.

Instructions:
1. Use the provided context (`context`) to understand relevant background (e.g. age, PSA levels, diagnosis).
2. Think step-by-step before answering, using medical reasoning (Chain of Thought).
3. Focus only on prostate cancer-related topics. If outside scope, say so politely.
4. Remind users to consult a real doctor for personal medical advice.

Respond using this format:

Step-by-step reasoning:
[Explain your medical reasoning based on the query and context]

Answer:
[Final, user-friendly and empathetic response]

---
Query: {user_input}
Context: {context}
"""
```

Figure 23

Figure 23 above shows the prompt engineering techniques initially implemented for both BioGPT and LLAMA. A use of role-playing, CoT reasoning and clear formatting was ensured to optimize the models response for the patient query whilst at the same time using the context provided from our retrieval model. This would've been a clear implementation and use of RAG for our model due to its generative response potential whilst using retrieved factual and relevant information.

However, a major issue arose when working with LLAMA in addition to prompt engineering techniques. Initially, the response times for LLAMA for a basic generation response was relatively poor at approx. 20 seconds however with the addition of these prompt engineering techniques further increased this whereby responses would take anywhere from 1-3mins to generate. After iterative adjustments and simplifications towards the prompt it seemed as though simply feeding a prompt to the LLAMA model was directly going to increase response time dramatically. Although the responses generated were majority relevant and suitable, the cost of response time was something that couldn't be ignored therefore focus turned to BioGPT.

Implementing this prompt into BioGPT caused similar issues due to the models small nature in size. With 347M parameters and a demand to generate appropriate responses as well as compute and follow the moderately extensive prompt, it was showing to be a difficulty for

BioGPT. Response times would average 10-15 seconds however the quality of response would be poor due to frequent hallucinations or incoherent sentences. The use of context also seemed to further confuse the model as the model couldn't generalise in case the query was medically related, due to classification from Section 3.6, but the retrieved context was prostate cancer specific. This meant RAG was unfortunately not attainable with BioGPT compared to LLAMA and simplification of the prompt needed to be executed to facilitate for BioGPT's size to have a balance of prompt engineering and generative reliability.

```
prompt = f"""As a clinical AI assistant specializing in oncology, respond to this medical query:

Query: {user_input}

Guidelines:
1. Provide accurate, evidence-based information
2. Acknowledge when queries are outside prostate cancer focus
3. Recommend consulting a physician for personal medical advice

Response:"""
```

Figure 24

Figure 24 shows the adjustments made to the prompt to try to best incorporate logical steps through CoT whilst also ensuring implementation of role-playing. This adjusted and simple prompt had shown greater success with responses and generation, so focus swiftly shifted to other objectives within the project.

3.6 Classification Model Implementation

A key concern was that the model wouldn't compute and respond to the user letting them know their query isn't medically relevant even with prompt information fed into the model addressing it. Therefore, implementation of a classifier model was done to appropriately categorize queries into medical and non-medical.

```
classifier = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")

def is_medical(query):
    candidate_labels = ["medical", "non-medical"]
    result = classifier(query, candidate_labels, multi_label=False)
    return result['labels'][0] == 'medical' and result['scores'][0] > 0.65
```

Figure 25

Figure 25 shows the initialization of BART-large-mnli and a function used to determine a queries relevancy based on the labels "medical" and "non-medical". The function would then return the result informing the chatbot whether the query is medically relevant.

```python
def generate_response(user_input, threshold=0.75):
    input_embedding = embedding_model.encode(user_input, convert_to_tensor=True)
    cos_scores = util.pytorch_cos_sim(input_embedding, question_embeddings)[0]

    top_score = torch.max(cos_scores).item()
    top_idx = torch.argmax(cos_scores)
    context = answers[top_idx]

    if top_score > threshold:
      return context

    if not is_medical(user_input):
        return "I specialize in prostate cancer information. Please ask medical questions related to this field."


    prompt = f"""As a clinical AI assistant specializing in oncology, respond to this medical query:

    Query: {user_input}

    Guidelines:
    1. Provide accurate, evidence-based information
    2. Acknowledge when queries are outside prostate cancer focus
    3. Recommend consulting a physician for personal medical advice

    Response:"""
```

Figure 26

Figure 26 shows where the function is called within the generate_response function as well as the appropriate chatbot response for when the query isn't medically relevant. This allowed and eased the demand on the BioGPT model so that it could focus on generating responses for relevant user queries which saves time as well as computational power.

3.7 Reinforcement Learning - Human Feedback

3.7.1 Database Creation

For continuous improvement of the model within the scope of the project as well as expansion, RLHF was partially implemented. Figure 27 below shows a function ran before the code is begun to create a database within Drive if not already created.

```python
# Create feedback.db database if not already created
def init_feedback_db():
    db_path = f'{drive_path}/feedback.db'
    conn = sqlite3.connect(db_path)
    c = conn.cursor()
    c.execute('''CREATE TABLE IF NOT EXISTS feedback
                 (id INTEGER PRIMARY KEY AUTOINCREMENT,
                  input_text TEXT,
                  response_text TEXT,
                  feedback TEXT,
                  timestamp DATETIME)''')
    conn.commit()
    conn.close()

init_feedback_db()
```

Figure 27

This database will store the following information: user query, chatbot response, user feedback and time. The feedback received by the user is in the form of a "thumbs up" or

“thumbs down which is displayed in Figure 28 and then a Thank you note is shown as displayed in Figure 29.

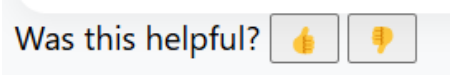


Figure 28

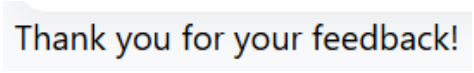


Figure 29

Being able to store this information within Drive allows the data to be stored permanently compared to if it was stored in cache, whereby every time Google Colab’s session would be restarted the data would be deleted. This is incredibly important since the model will require time and large amounts of data to improve and learn to formulate better responses therefore the permanent storing and continuous accumulation of data over time is able to successfully facilitate this requirement.

3.7.2 HTML Adjustments

To ensure simple and functioning transfer of information between frontend and backend, the HTML template required an additional section incorporating the ability to receive and store user feedback as well as other information alongside it. Figure 30 displays the simple addition of the feedback buttons within the UI and Figure 31 displays in detail the POST methods added to successfully retrieve user_input and store it for later upload into the database.

```
<div class="feedback">
<span>Was this helpful?</span>
<button onclick="submitFeedback('thumbs_up')">👍</button>
<button onclick="submitFeedback('thumbs_down')">👎</button>
```

Figure 30

```
<script>
    function submitFeedback(type) {
        const userInput = document.querySelector('input[name="user_input"]').value;
        const responseText = document.querySelector('.response p').innerText;

        fetch('/feedback', {
            method: 'POST',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify({
                input: userInput,
                response: responseText,
                feedback: type
            })
        });

        document.querySelector('.feedback').innerHTML = 'Thank you for your feedback!';
    }
    </script>
```

Figure 31

3.7.3 Secondary Back End Point

Figure 32 displays the secondary app.route method that was implemented to ensure the seamless retrieval of information from the front end into the back end. SQLite3 is utilized to connect to our feedback.db database whereby c.execute then enables the retrieval and addition of the relevant information into the database for permanent and safe storage.

To further improve reliability and error handling an exception was created to signal when the system is having trouble storing feedback into the database.

```
@app.route("/feedback", methods=["POST"])
def handle_feedback():
    data = request.json
    try:
        db_path = f'{drive_path}/feedback.db'
        conn = sqlite3.connect(db_path)
        c = conn.cursor()
        c.execute('''INSERT INTO feedback
                     (input_text, response_text, feedback, timestamp)
                     VALUES (?, ?, ?, ?)''',
                     (data['input'], data['response'],
                      data['feedback'], datetime.now()))
        conn.commit()
        return jsonify(success=True)
    except Exception as e:
        print(f"Error saving feedback: {e}")
        return jsonify(success=False, error=str(e)), 500
    finally:
        conn.close()
```

Figure 32

To further add functionality, Figure 33 shows a function which allows quick access and viewing of the latest 10 database entries including query, response, feedback and time. The length of data displayed is also adjustable within tail(…).

```
# View last 10 queries and their feedback
def view_feedback():
    db_path = f'/content/drive/My Drive/MedBot_Data/feedback.db'
    conn = sqlite3.connect(db_path)
    df = pd.read_sql("SELECT * FROM feedback", conn)
    conn.close()
    return df

view_feedback().tail(10)
```

Figure 33

4 Testing

A variety of testing strategies were evaluated to best display the performance and usability of the chatbot.

4.1 Manual Testing

4.1.1 Prostate Cancer Queries

Initial testing was conducted through a variety of individual prostate cancer specific queries, with high similarity to a specific query stored in data.csv, was conducted. Slight rephrasing and re-wording were implemented to test the model's reliability at assessing the relevance of the user query.
Fortunately, every input of slight rephrased or re-worded queries resembling already stored FAQ's, managed to successfully retrieve and output the relevant answers and responses. Moreover, these responses were delivered very promptly in under 1-2 seconds with no delay satisfying the objective of the chatbot being fast with responsiveness.

Despite this a clear withdrawal identified was that due to the embedding model scoring embeddings based on word similarity. If a prostate cancer related query was given which included several key words of an already stored FAQ however wasn't of the same concern, the model would still respond with the FAQ stored information.
An example of this is displayed in Figure 34 below. This may be an occurrence due to our generative model's small parameter size. If a larger model was to be implemented, it would be able to compute larger prompts as well as implement RAG. This would allow the ability to better differentiate between the user query and the retrieved information thus leading to more directed and informed information tailored to the users concerns.

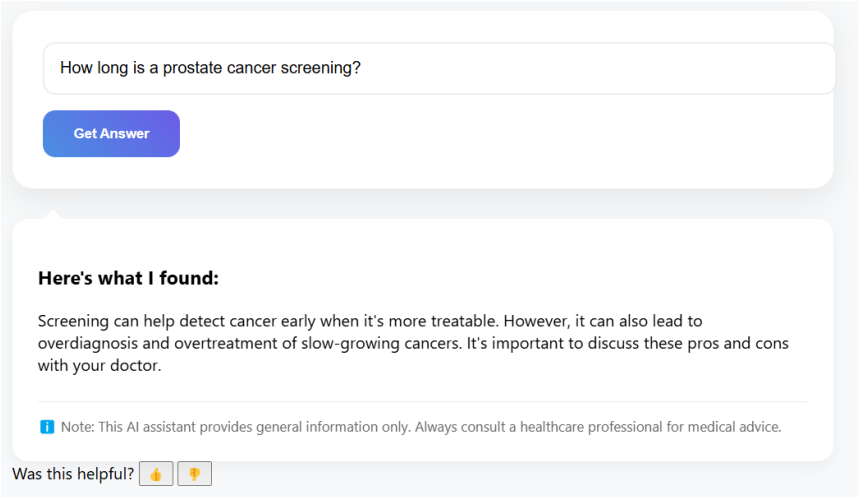


Figure 34

4.1.2 Medical Queries

Secondary testing was conducted on a variety of medically related or slight PC related queries. Testing reinforced that the model's size was still a clear drawback within our chatbot. Many general medical queries generated a suitable and appropriate response, however many didn't include enough information or consultation. This is expected as our model is specialized on

PC information however the generative model was trained on a vast amount of medical information and additional functionality of general medical information was considered. A larger general-purpose model would've been able to appropriately facilitate the need for specialized PC queries as well as general medical enquiries through techniques such as RAG and suitable prompting.
Nevertheless, the model was able to still appropriately deliver responses and information to many queries within an approximate 5 seconds of query submission.

4.1.3 Non Medical Queries

Fortunately, the model was appropriately able to provide a response stating it specializes in medical PC information for every enquiry which wasn't classified as medically relevant. These responses were delivered very promptly.

4.2 Peer Testing

To eliminate bias, I had also conducted testing through friends and family. I had let 4 individuals test my chatbot through a variety of questions assuming they were looking for PC related information. Each was given the ability to ask 20 questions and then required to submit their feedback for every query. The table in Figure 35 below shows the information gathered from testing.

Peer Feedback Assessment

User No.	Positive Feedback	"Have you learnt new information about PC?"
1	16/20	Yes
2	18/20	Yes
3	12/20	Yes
4	15/20	Yes

Figure 35

Results proved that our model was able to achieve average positive feedback of 76.25% for every enquiry, with all our individuals stating they've learnt something new about prostate cancer. These results though positive may not be an accurate depiction of the quality of the chatbot responses. Testing with medical professionals would be required to rigorously evaluate and improve the chatbot to high medical standards and guidelines.

5 Critical Evaluation

Overall, this chatbot along with its features had its clear strengths and drawbacks which could be resolved with further iterative improvement and expansion.

With a possible expansion of the project, including a larger model, a greater variety of topics could've been addressed, leading to the potential implementation of frameworks such as LangChain, which would've served as a strong addition by providing built-in memory and context handling, allowing for more personalized and coherent conversations in order to further improve patient engagement as previous conversations could be more easily referenced and elaborated upon. In a broader scope, LangChain could have been integrated to orchestrate the flow of conversation as well as manage multi-step reasoning and connect the chatbot to a vector database for retrieval-augmented generation or even a potential calendar API for appointment handling. This would have enabled the system to function not just as a responder but as an assistant capable of recalling patient-specific information, tracking context over time and performing specific actions. LangChain's agent-based model and tool integrations would have added a scalable layer of intelligence beyond pure text generation, helping the chatbot act with memory and awareness across longer, more complex conversations.

Furthermore, the integration of RAG could've been used to significantly enhance the chatbot's ability to provide accurate, up-to-date, and context-specific information. Instead of relying solely on the static knowledge embedded within the model and the database provided, a RAG setup would allow the chatbot to retrieve relevant information from a curated knowledge base and craft an appropriate response. This retrieval step would feed contextual data directly into the model before generating a response, making answers more grounded and reliable. For a healthcare chatbot, this could be particularly valuable when dealing with medical terminology or treatment options ensuring that responses are both informed and safe. RAG would also allow for the easy updating of knowledge without retraining the entire model, providing scalability and adaptability as new medical information becomes available or as institutional policies change.

Moreover, the use of ngrok ensured the URL was temporary and reset each time the tunnel was reset, making the model unsuitable for long-term or production use. Additionally, the free tier can be unstable and lacks scalability or proper authentication, which would be crucial for a medically sensitive application. The integration of the chatbot into a more robust or publicly accessible tool would need to be executed into cloud platforms like AWS, Azure etc. and integrated to HTTPS with a custom domain and ensure it's scalable, reliable and compliant with healthcare data standards with user authentication and other data privacy compliances.

Although considered, unfortunately implementation of a possible patient login could not be addressed within the scope of this project due to limited time. The addition of this feature was of great interest and would've even better addressed the main objectives of this project. With individual patient login, data privacy implementations would need to be addressed. This addition would allow the continuous addition of personal data and information for the model to work from and tailor responses for the individual.

Despite these limitations due to the scope of the project, the primary objectives of the project were successfully met. The demand for fast responsiveness, accurate medical information and

positive feedback towards PC education and insight was met. With AI moving rapidly through our economy and workplace and soon to be integrated into many regions and sectors, there is a implementation of this new technology within the bounds of healthcare. AI chatbot use for both patients and doctors is a developing process and technology which has the potential to massively improve and leave a lasting positive impact in the healthcare industry. With demand for constant iteration and improvement with models, prompts, training, data and more, it's clear that there's so much potential which we are yet to uncover.

6 References

[1] Stanford Report. (2025). *Study suggests physicians make better decisions with help of AI chatbots*.
https://news.stanford.edu/stories/2025/02/study-suggests-physician-s-medical-decisions-benefit-from-chatbot

[2] Menz, B. D., Modi, N. D., Abuhelwa, A. Y., Ruanglertboon, W., Vitry, A., Gao, Y., ... & Hopkins, A. M. (2025). *Generative AI Chatbots for Reliable Cancer Information: Evaluating web-search, multilingual, and reference capabilities of emerging large language models. European Journal of Cancer*,

[3] Chen, S., Kann, B. H., Foote, M. B., Aerts, H. J., Savova, G. K., Mak, R. H., & Bitterman, D. S. (2023). *Use of artificial intelligence chatbots for cancer treatment information. JAMA oncology*, *9*(10), 1459-1462.

[4] Ashley M Hopkins, Jessica M Logan, Ganessan Kichenadasse, Michael J Sorich, *Artificial intelligence chatbots will revolutionize how cancer patients access information: ChatGPT represents a paradigm-shift*, *JNCI Cancer Spectrum*, Volume 7, Issue 2, April 2023, pkad010,

[5] Görtz M, Baumgärtner K, Schmid T, et al. *An artificial intelligence-based chatbot for prostate cancer education: Design and patient evaluation study*. DIGITAL HEALTH. 2023;9.

[6] Malec, M. (2024, December 2). *Mastering LLM integration: 6 steps every CTO should follow*. HatchWorks.
https://hatchworks.com/blog/gen-ai/llm-integration-guide/

[7] Gaper.io. (2024, June 17). *Building smarter apps: Integrating LLM APIs for seamless natural language understanding*.
https://gaper.io/smarter-apps-integrating-llm-apis/

[8] Government Digital Service. (2025, April 22). *AI Insights: Prompt engineering*. GOV.UK.
https://www.gov.uk/government/publications/ai-insights/ai-insights-prompt-engineering-html

[9] Hugging Face. (2023). *microsoft/biogpt*.
https://huggingface.co/microsoft/biogpt

7 Appendices

A. Use of Third-Party Code, Libraries and Generative AI

Libraries:

```
from flask import Flask, request, render_template_string, jsonify
import torch
from transformers import BioGptTokenizer, BioGptForCausalLM
from transformers import pipeline, set_seed, AutoTokenizer,
AutoModelForSequenceClassification
from pyngrok import ngrok
from sentence_transformers import SentenceTransformer, util
import pandas as pd
import sqlite3
from datetime import datetime
import os
from google.colab import drive
```

Other code/software:

- Online HTML template (unable to locate)
- Notion (software used to organize my work progression)